

# Deployment Guide

## Rotana Spa Management System

### 1. Server requirements

| Resource      | Minimum                                            | Recommended                |
|---------------|----------------------------------------------------|----------------------------|
| Node.js       | 20.x LTS                                           | 22.x LTS                   |
| RAM           | 1 GB                                               | 2 GB+                      |
| CPU           | 1 vCPU                                             | 2 vCPU                     |
| Disk          | 2 GB free                                          | 5 GB+ (logs, .next )       |
| Database      | PostgreSQL 15+ (managed Neon, RDS, or self-hosted) | same                       |
| Reverse proxy | —                                                  | Nginx / Caddy / Cloudflare |

### 2. Environment variables ( .env )

```
DATABASE_URL=postgresql://user:password@host:5432/dbname?sslmode=require
JWT_SECRET=<long-random-secret-at-least-32-characters>
# optional
PORT=3000
NODE_ENV=production
```

- `DATABASE_URL` — node-postgres connection string. Use a **least-privilege** app role (no superuser).
- `JWT_SECRET` — used to sign auth tokens. Generate with `openssl rand -hex 32` . Rotate on personnel change.
- Never commit `.env` ; it is already ignored via `.gitignore` .

### 3. Database installation

```
# 1. create the database and role (as postgres superuser / via provider console)
CREATE ROLE spa_app LOGIN PASSWORD '...';
CREATE DATABASE spa_management OWNER spa_app;

# 2. apply migrations IN ORDER
psql "$DATABASE_URL" -f db-migration.sql
psql "$DATABASE_URL" -f db-migration-v2.sql
# ... continue v3 .. v32 ...
```

```
psql "$DATABASE_URL" -f db-migration-v33.sql
psql "$DATABASE_URL" -f db-migration-v34.sql
```

Alternative: `node scripts/run-migrations.js` after setting `DATABASE_URL`.

## 4. Build & start (bare Node)

---

```
npm ci
npm run build
NODE_ENV=production npm run start    # default port 3000
```

Health check: `curl http://localhost:3000/api/companies/public`.

## 5. Process manager (PM2)

---

An `ecosystem.config.js` is included:

```
npm i -g pm2
pm2 start ecosystem.config.js
pm2 save
pm2 startup    # enable boot persistence
pm2 logs rotana-spa
```

## 6. Reverse proxy (Nginx example)

---

```
server {
    listen 80;
    server_name spa.example.com;
    location / {
        proxy_pass http://127.0.0.1:3000;
        proxy_http_version 1.1;
        proxy_set_header Host $host;
        proxy_set_header X-Real-IP $remote_addr;
        proxy_set_header X-Forwarded-For $proxy_add_x_forwarded_for;
        proxy_set_header X-Forwarded-Proto $scheme;
    }
}
```

Add TLS (Let's Encrypt / certbot) and force HTTPS.

## 7. Hardening checklist

---

- ☐ HTTPS enforced (redirect HTTP).
- ☐ `NODE_ENV=production` ; dev server not exposed publicly.
- ☐ Least-privilege DB role; app never uses `postgres / superuser` .
- ☐ Strong `JWT_SECRET` ; not in version control.
- ☐ `npm audit` clean and dependencies pinned.
- ☐ Database backups enabled (PITR) + weekly snapshot.
- ☐ Rate limiting / fail2ban on the proxy.
- ☐ Only essential ports open (443, SSH).

## 8. Backup

---

- **DB:** provider-level continuous backups + daily snapshot; restore-tested quarterly (see `SECURITY.md` §8).
- **App:** code is version-controlled (git); config lives in `.env` (back it up separately and securely).
- **Recovery script:**

```
# dump
pg_dump "$DATABASE_URL" -f backup-$(date +%F).sql
# restore to a fresh db
psql "$DATABASE_URL_NEW" -f backup-YYYY-MM-DD.sql
```

## 9. Update / deploy process

---

1. `git pull` (or CI checkout) the target release.
2. `npm ci` (exact lockfile install).
3. If new migrations exist: apply them **before** starting the new build (in `vN` order).
4. `npm run build` .
5. Restart the process: `pm2 reload rotana-spa` (or systemd restart).
6. Smoke test: login, dashboard stats, one visit + service-order flow, one report.
7. On failure, roll back: redeploy previous release and restore DB to pre-migration snapshot (only if a migration caused the issue).

## 10. Vercel (alternative hosting)

---

This app is Vercel-compatible:

```
vercel env add DATABASE_URL      # production
vercel env add JWT_SECRET       # production
vercel --prod
```

Note: serverless connection pooling is recommended for the Neon/Postgres connection ( `pg` pool with `sslmode=require` ).

# 11. Maintenance tasks

---

| Task                          | Cadence               |
|-------------------------------|-----------------------|
| DB backup + restore test      | quarterly             |
| <code>npm audit</code> review | monthly               |
| Log review                    | weekly                |
| User/role review              | monthly               |
| Demo license expiry review    | monthly (super admin) |